

一、余弦相似度 & 余弦距离

要分析不同歌曲之间的相似度，我们需要将这两种截然不同的格式（MP3 音频和 MIDI 乐谱）分开处理，提取各自的特征，然后再计算相似度。

- **MP3（音频层）**：包含真实的波形、音色和后期效果。我们将提取 **MFCC（梅尔频率倒谱系数，代表音色）** 和 **Chroma（色度图，代表和弦与调性）**。
- **MIDI（符号层）**：只包含音符、时长和力度。我们将提取 **Pitch Class Histogram（音高类直方图，代表旋律和声学特征）**。

下面使用**余弦相似度**（Cosine Similarity）来对比歌曲特征（旋律走势）是否相似：

$$\text{余弦相似度} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

由于需要比较的两个音乐序列中的值都非负，所以 $\mathbf{A} \cdot \mathbf{B} \geq 0$ 。又根据柯西不等式，则余弦相似度介于 0-1 之间。

余弦相似度表示两个特征向量之间的夹角余弦值，值越远离 0 越接近 1，则两段音乐越相似。

余弦距离的定义：余弦距离 $d = 1 - \text{余弦相似度}$

余弦距离的值越远离 1 越接近 0，则两段音乐越相似。

二、动态时间规整（DTW）

引入 DTW（Dynamic Time Warping，动态时间规整）是一个非常专业的决定。前面的全局平均法就像是在比较两首歌的“成分表”，而 DTW 则是真正让两首歌在时间轴上“肩并肩”地走一遍，看看它们的旋律和声走向是否一致。

DTW 的核心数学思想是寻找一条最优匹配路径（Warping Path），使得两个不同长度序列 X 和 Y 之间的累计距离最小：

$$DTW(X, Y) = \min_W \sum_{k=1}^K d(w_{xk}, w_{yk})$$

算法公式说明 1

这种算法极其适合音乐查重，因为它**允许速度的拉伸和压缩**——即使一首歌唱得快、一首歌唱得慢，只要旋律相似，DTW 都能将其完美对齐。

1. 基础对象

- X 和 Y ：
 - **数学含义**：代表两个长度可能不同的时间序列。
 - **音乐含义**：代表你要对比的两首歌（比如歌曲 A 和歌曲 B）提取出来的 **Chroma 特征序列**。因为歌手演唱速度不同，这两首歌的时间帧数（长度）可能是不一样的。

2. 核心变量：路径 W

- W (Warping Path):
 - **数学含义**：规整路径，也就是序列 X 和 Y 之间的一种“对齐方式”。
 - **音乐含义**：你可以把它想象成一个“指挥家”。为了让两首歌的节拍对齐，指挥家会让第一首歌在某一句唱快一点，在另一句唱慢一点。 W 就是这首拉伸、压缩的**对齐时间轴**。
- $\min_W$ (Minimize over W):

- **数学含义**：在所有可能的路径 W 中，寻找让后面那个求和公式结果**最小**的那一条。
- **音乐含义**：这是 DTW 的“灵魂”。两首歌有无数种对齐方式（比如瞎对齐），但算法会遍历所有可能，找到“错位最少、最完美同步”的那一种对齐方案。

3. 路径的具体拆解

- K :
 - **数学含义**：最优对齐路径 W 的**总长度（总步数）**。
 - **音乐含义**：对齐这两首歌总共需要走多少个“时间步”。
- k :
 - **数学含义**：路径上的当前步数序号（从 1 一直走到 K ）。
- w_{xk} 与 w_{yk} :
 - **数学含义**：在对齐路径的第 k 步时，序列 X 走到了哪个数据点，序列 Y 走到了哪个数据点。
 - **音乐含义**：比如在第 50 步对齐时，歌曲 A 播放到了第 12 秒 (w_{xk})，而歌曲 B 播放到了第 14 秒 (w_{yk})。DTW 强行把这两秒拉到了一起对比。

4. 距离计算

- $d(\cdot, \cdot)$ (Distance):
 - **数学含义**：余弦距离 d ,两个数据点之间的“局部距离”或“代价”。
 - **音乐含义**：余弦距离 d 计算的是，歌曲 A 的第 12 秒和歌曲 B 的第 14 秒，它们在这个瞬间的**和弦走向有多大的差异**。和弦越像，代价 d 越接近 0。
- $\sum_{k=1}^K$ (Summation):
 - **数学含义**：把路径上每一步的局部距离 d 累加起来。
 - **音乐含义**：沿着这条最优的对齐时间轴，把每一步的“差异度”全加起来，就得到了这两首歌在采用最佳对齐状态下的**总差异代价（Total Cost）**。总代价越小，说明这两首歌越相似。

小结：在时间不能倒流的前提下 ($\min_W$)，我们拉伸或压缩两首歌的速度，让它们从头到尾对齐 ($\sum_{k=1}^K$)。当我们找到一种让每一瞬间的和弦差异 (d) 加起来总和最小的对齐方式时，这个最小的总差异，就是这两首歌的 DTW 距离。

算法公式说明 2

对于两个长度分别为 N 和 M 的时间序列，标准 DTW 的时间复杂度为 $O(N \times M)$ ，空间复杂度为 $O(N \times M)$ 。

以下是严谨的数学推导与证明过程：

1. 算法数学定义与状态转移方程

假设我们有两个时间序列：

- 序列 $X = (x_1, x_2, \dots, x_N)$ ，长度为 N
- 序列 $Y = (y_1, y_2, \dots, y_M)$ ，长度为 M

DTW 的目标是构建一个 $N \times M$ 的累计距离矩阵 (Accumulated Cost Matrix) D 。矩阵中的每一个元素 $D[i, j]$ 代表从起点 $(1, 1)$ 到达当前点 (i, j) 的最小累计代价。

根据动态规划的局部最优原理，DTW 的核心状态转移方程 (Bellman Equation) 定义为：

$$D[i, j] = d(x_i, y_j) + \min \begin{cases} D[i-1, j] & \text{(插入/垂直移动)} \\ D[i, j-1] & \text{(删除/水平移动)} \\ D[i-1, j-1] & \text{(匹配/对角线移动)} \end{cases}$$

其中：

- $d(x_i, y_j)$ 是两个样本点之间的局部距离（例如我们之前代码里用的余弦距离）。
- 边界条件初始化为： $D[0,0] = 0$ ，且对于 $i > 0, j > 0$ 有 $D[i, 0] = \infty, D[0, j] = \infty$ 。

2. 时间复杂度证明： $O(N \times M)$

动态规划的时间复杂度 = 状态空间的总数 $\times$ 计算单个状态的代价。

1. 状态空间的总数：

矩阵 D 的维度是 $N \times M$ 。为了得到最终结果 $D[N, M]$ ，算法必须遍历并计算矩阵中除了初始化边界外的每一个单元格 (i, j) 。因此，总共需要计算的状态数量是 $N \times M$ 个。

2. 计算单个状态的代价：

在计算某一个具体的 $D[i, j]$ 时，需要执行两步操作：

- 计算局部距离 $d(x_i, y_j)$ ：对于确定维度的特征向量（比如 12 维的 Chroma），这需要常数级的时间，记为 $O(1)$ 。
- 查找前驱节点的最小值 $\min(D[i-1, j], D[i, j-1], D[i-1, j-1])$ ：这是在 3 个已知的标量中找最小值，操作次数固定，记为 $O(1)$ 。
- 相加并赋值： $O(1)$ 。

因此，计算单个格子的总代价是常数级 $O(1)$ 。

结论：总时间复杂度 = $(N \times M) \times O(1) = O(N \times M)$ 。

如果两首歌的长度相近 ($N \approx M$)，则时间复杂度呈二次方级 $O(N^2)$ 。

3. 空间复杂度证明： $O(N \times M)$

空间复杂度取决于算法运行过程中需要在内存中同时驻留多少数据。

1. 标准实现（求解对齐路径）：

在音乐查重中，我们不仅需要最终的相似度得分 $D[N, M]$ ，还需要知道两首歌是怎么对齐的（即回溯寻找 Warping Path wp）。

为了能够从右下角反向推导回左上角，系统必须把完整的 $N \times M$ 累计矩阵保存在内存中。

结论：空间复杂度为 $O(N \times M)$ 。

2. 极限优化版（仅求相似度距离）：

如果你不需要提取对齐路径（不回溯），仅仅为了得到最终的那个总分 $D[N, M]$ 。观察状态转移方程可以发现，计算第 i 行的数据，只依赖于第 $i-1$ 行，之前的行全都可以丢弃。在这种空间优化下，内存中只需要维护两行（当前行和上一行），每行长度为 M 。

结论：优化后的空间复杂度为 $O(\min(N, M))$ 。但在之前的 Python 代码中，因为我们用 librosa 获取了 wp 路径，所以其实际占用依然是 $O(N \times M)$ 。

4. 工程中的约束优化（为什么 $O(N^2)$ 是个灾难）

在 $O(N^2)$ 复杂度下，如果不做降采样，一首 3 分钟的音频（采样率 22050Hz，帧长 512）

会产生约 7700 帧。两首歌对比，矩阵大小就是 $7700 \times 7700 \approx 5900$ 万个格子。计算

$N \times N$ 首歌曲的矩阵会导致系统直接崩溃。

这也是为什么在主流算法库中，通常会引入全局路径约束（Global Path Constraints），最著名的是 Sakoe-Chiba Band 或 Itakura Parallelogram：

- 原理：强制规定对齐路径不能偏离矩阵对角线太远（限制最大时间差偏移量为 w ）。
- 优化结果：算法不需要计算整个矩阵，只需要计算对角线两侧宽度为 w 的带状区

域。时间复杂度直接从 $O(N^2)$ 骤降至 $O(N \times w)$ ($w \ll N$)。

三、结果与意见

对 N 首歌两两进行 DTW 相似度比较，并输出 $N \times N$ 矩阵，由于矩阵对称，只需计算上三角矩阵。具体地，任意第 i, j 两首歌之间的总相似度定义为下：

$$\text{总相似度}_{ij} = 1 - \frac{DTW(M_i, M_j)}{K}$$

结果表示两首歌之间的总相似度，公式为 1-按对齐总步数等权平均的余弦距离，由上述数理推导可证明值介于 0-1 之间，值越远离 0 越接近 1，则两段音乐越相似。

100.00%	62.35%	75.30%	63.99%	77.37%	49.68%	64.93%	62.08%	59.32%	68.82%	73.65%	67.27%	79.64%	58.74%	54.70%	68.64%	68.82%	70.00%	69.78%
62.35%	100.00%	68.01%	69.22%	63.25%	74.05%	61.32%	70.39%	71.09%	58.90%	60.75%	63.89%	77.31%	75.72%	72.77%	62.80%	67.54%	54.33%	59.08%
75.30%	68.01%	100.00%	72.30%	74.97%	68.97%	72.79%	70.06%	70.50%	67.97%	72.80%	75.20%	82.37%	75.54%	69.07%	74.79%	73.21%	72.17%	70.24%
63.99%	69.22%	72.30%	100.00%	61.23%	70.62%	54.13%	78.07%	70.56%	54.38%	58.94%	53.11%	67.49%	77.89%	68.36%	71.54%	56.78%	62.55%	74.78%
77.37%	63.25%	74.97%	61.23%	100.00%	60.55%	63.35%	57.24%	57.43%	66.17%	72.52%	64.74%	73.91%	59.30%	47.38%	66.41%	61.09%	68.17%	67.11%
49.68%	74.05%	68.97%	70.62%	60.55%	100.00%	60.41%	68.07%	70.65%	55.63%	60.54%	55.43%	67.87%	77.04%	75.51%	67.29%	61.26%	60.21%	65.42%
64.93%	61.32%	72.79%	54.13%	63.35%	60.41%	100.00%	54.92%	59.74%	76.07%	66.83%	76.64%	77.55%	61.50%	58.96%	61.82%	68.94%	61.04%	54.92%
62.08%	70.39%	70.06%	78.07%	57.24%	68.07%	54.92%	100.00%	70.48%	57.57%	58.76%	60.27%	65.20%	78.66%	69.04%	66.25%	54.98%	65.41%	73.35%
59.32%	71.09%	70.50%	70.56%	57.43%	70.65%	59.74%	70.48%	100.00%	50.12%	59.86%	46.12%	64.21%	75.97%	69.55%	65.49%	58.19%	60.36%	70.37%
68.82%	58.90%	67.97%	54.38%	66.17%	55.63%	76.07%	57.57%	50.12%	100.00%	68.64%	77.27%	77.32%	55.29%	48.38%	60.54%	64.53%	63.89%	60.31%
73.65%	60.75%	72.80%	58.94%	72.52%	60.54%	66.83%	58.76%	59.86%	68.64%	100.00%	66.99%	75.11%	64.26%	52.62%	65.62%	55.00%	67.65%	68.61%
67.27%	63.89%	75.20%	53.11%	64.74%	55.43%	76.64%	60.27%	46.12%	77.27%	66.99%	100.00%	81.67%	60.52%	50.24%	67.76%	64.72%	62.50%	58.48%
79.64%	77.31%	82.37%	67.49%	73.91%	67.87%	77.55%	65.20%	64.21%	77.32%	75.11%	81.67%	100.00%	71.94%	64.56%	74.85%	70.40%	70.71%	72.15%
58.74%	75.72%	75.54%	77.89%	59.30%	77.04%	61.50%	78.66%	75.97%	55.29%	64.26%	60.52%	71.94%	100.00%	73.92%	75.58%	61.64%	68.47%	72.92%
54.70%	72.77%	69.07%	68.36%	47.38%	75.51%	58.96%	69.04%	69.55%	48.38%	52.62%	50.24%	64.56%	73.92%	100.00%	67.49%	60.23%	53.07%	64.42%
68.64%	62.80%	74.79%	71.54%	66.41%	67.29%	61.82%	66.25%	65.49%	60.54%	65.62%	67.76%	74.85%	75.58%	67.49%	100.00%	63.01%	68.60%	75.82%
68.82%	67.54%	73.21%	56.78%	61.09%	61.26%	68.94%	54.98%	58.19%	64.53%	55.00%	64.72%	70.40%	61.64%	60.23%	63.01%	100.00%	55.14%	52.86%
70.00%	54.33%	72.17%	62.55%	68.17%	60.21%	61.04%	65.41%	60.36%	63.89%	67.65%	62.50%	70.71%	68.47%	53.07%	68.60%	55.14%	100.00%	70.41%
69.78%	59.08%	70.24%	74.78%	67.11%	65.42%	54.92%	73.35%	70.37%	60.31%	68.61%	58.48%	72.15%	72.92%	64.42%	75.82%	52.86%	70.41%	100.00%

$N \times N$ 矩阵表显示，艺人海岛大猫咪一龙（我）公开发行的上述歌曲间总相似度大多超过 50%，并不能有效地区分每首歌曲间的音乐差异性和多样性，对我未来发行新版音乐帮助和意义不大。因此，我暂时弃用这个 DTW 音乐相似度的经典算法，若未来时机成熟，可加以改进。